

From a workflow to a pipeline you would trust

Yesterday you learned the parts. Today every team builds a real deploy pipeline, breaks it four times on purpose, fixes it, and ships to its own URL.

The capstone, solved

By hand vs pipeline

Four reds, four fixes

Your own URL

Rollbacks

DAY 2 AT A GLANCE

Build it, break it, fix it, ship it

Time	Part	What it covers
09:00	Welcome back	Day 1 in five lines
09:10	Yesterday's capstone, solved	Sentence, boxes, YAML, then the real run: red, locked, fixed
10:00	Breakfast	
10:30	From a laptop to a pipeline	Deploying by hand, then the pipeline you will build
11:15	Set up your team's repo	Template, seven variables, the production environment
11:40	Build it: four reds, four fixes	Each failure teaches one idea. Then a teammate approves
13:20	Rollbacks	Deployment patterns, and one-click rollback
14:00	Lunch	
15:00	Finish, prove it, roll it back	Your team's own URL, live
16:15	Kahoot	The long one, with screenshots

All the teaching is before lunch. After lunch you build. Every screen on these slides is from the repos you will use.

Everything today builds on these

- 1 An event starts a workflow, read from the commit that triggered it.
- 2 Every job is a brand new machine. Nothing on disk survives between jobs.
- 3 needs decides when. It never moves files.
- 4 Pin actions to a full commit SHA. Tags move.
- 5 A workflow reports. A branch protection rule enforces.

WARM UP · 09:10

From a flow to a file

Yesterday's capstone, solved slowly. First as a sentence, then as boxes, then as YAML, then as a file in a repo, and then the real run: one machine red, the merge locked, one line to fix it.

YOU WILL BE ABLE TO

- Turn a plain sentence into a flowchart
- Turn each box of the flowchart into one YAML key
- Put the file where GitHub will find it and run it
- Read a real run: which machine failed, and why

WATCH AND READ

- Capstone brief: [labs/capstone](#)
- The real run: github.com/Greyisheep/gh-200-capstone-demo
- The cheatsheet at the back of this deck
- Docs: Workflow syntax for GitHub Actions

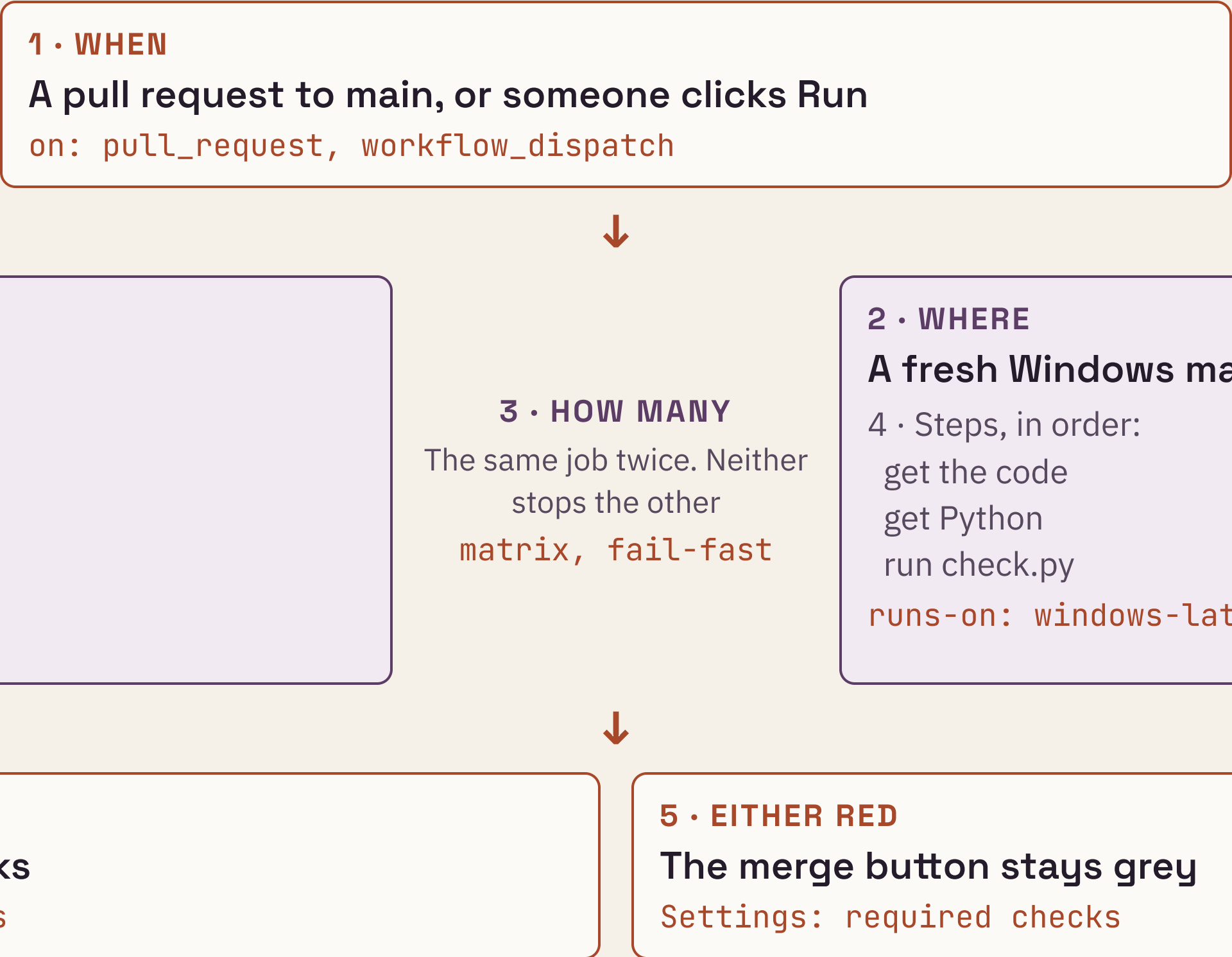
Start with a sentence, not with YAML

Before anyone merges into main¹, on Ubuntu and on Windows², let both finish even if one fails³, run check.py⁴, and refuse the merge if either is red⁵.

¹ When? The events that start it on:	² Where? The kind of machine runs-on:	³ How many? One copy, or several side by side strategy:	⁴ What steps? What happens on the machine, in order steps:	⁵ Who enforces? What stops a red result getting merged Settings
---	--	--	---	--

Every workflow you will ever write answers these five questions. If you cannot answer one in plain words, no amount of YAML will fix it.

Draw it as boxes



Box 1 becomes on:

1

When

on:

2

Where

runs-on:

3

How many

strategy:

4

What steps

steps:

5

Who enforces

Settings

```
name: PR gate
on:
  pull_request:
    branches: [main]
  workflow_dispatch:
```

- name: is for humans**

It is the title in the Actions tab. GitHub does not care what it says.
- on: is a list of events**

Two events here, so two keys under it. The order does not matter.
- workflow_dispatch: with nothing after it**

It really is empty. It adds the Run workflow button, and needs no settings.

Boxes 2 and 3 become the job

1
When
on:

2
Where
runs-on:

3
How many
strategy:

4
What steps
steps:

5
Who enforces
Settings

```
name: PR gate
on:
  pull_request:
    branches: [main]
  workflow_dispatch:
jobs:
  check:
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
    runs-on: ${{ matrix.os }}
```

check is a name you chose
It is the job's id. It shows up as the check name on the PR, so pick one you are happy to read.

matrix makes the copies
One list, two values, two jobs. Each copy gets one value in matrix.os.

runs-on reads that value
\${{ matrix.os }} is filled in per copy: ubuntu-latest in one, windows-latest in the other.

Box 4 becomes the steps

- 1

When

on:
- 2

Where

runs-on:
- 3

How many

strategy:
- 4

What steps

steps:
- 5

Who enforces

Settings

```
name: PR gate
on:
  pull_request:
    branches: [main]
  workflow_dispatch:
jobs:
  check:
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@<SHA> # v7.0.1
      - uses: actions/setup-python@<SHA> # v7.0.0
        with:
          python-version: '3.12'
      - name: Run the check
        run: python capstone/check.py
```

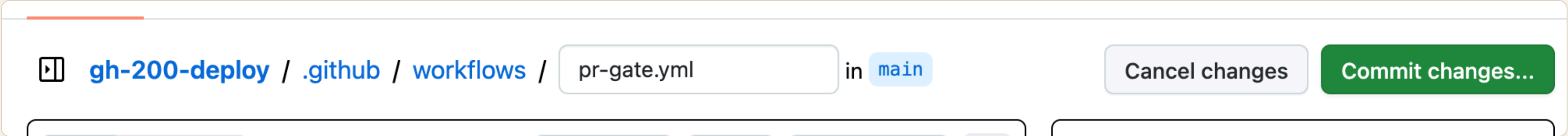
- Each dash is one step

Four lines starting with a dash, three steps. Lines without a dash belong to the step above.
- uses: borrows, run: types

Two borrowed actions to set the machine up, then one command of our own.
- with: feeds the action

python-version is an input setup-python asks for. Quoted, so YAML keeps it as text.

Move it into a file GitHub will find



Real screen: typing the path turns each slash into a folder, shown in the breadcrumb.

Five moves

- 1. Make a branch. Never build a gate straight on main
- 2. Add file, Create new file, and type **.github/workflows/pr-gate.yml**. The slashes make the folders
- 3. Paste the YAML, commit to the branch
- 4. Open a pull request to main. The workflow runs straight away
- 5. Watch two checks appear on the pull request, one per OS

What you wrote	Where it shows up
name: PR gate	The Actions tab list
job id check + matrix value	Check name check (ubuntu-latest)
name: Run the check	The step's line in the log
file name pr-gate.yml	Only the file path

Two machines, one red

Summary

All jobs

check (ubuntu-latest)

1 check (windows-latest)

Run details

Usage

Workflow file

Triggered via pull request 4 minutes ago

Greyisheep opened #1 [add-pr-gate](#)

Status

Failure

Total

17s

pr-gate.yml

on: pull_request

Matrix: check

2 jobs completed

Show all jobs

Annotations

1 error and 1 notice

2 check (windows-latest)

Process completed with exit code 1.

- 1 check (windows-latest)
Red. Ubuntu, the same code, is green.
- 2 Process completed with exit code 1
The symptom. The cause is the line above it in the log:
- The real line
Path mismatch:
'data\\reports\\summary.txt' is not
'data/reports/summary.txt'

Red means the merge button stays grey

Some checks were not successful

1 failing, 1 successful checks

1 failing check

✖

1

PR gate / check (windows-latest) (pull_request) Fa...

Required

⋮

1 successful check

✓

2

PR gate / check (ubuntu-latest) (pull_request) Suc...

Required

⋮

Merge pull request

You can also merge this with the command line.
[View command line instructions.](#)

- 1

Required, and failing

The ruleset lists both matrix legs by name. One is red.
- 2

Merge pull request

Greyed out. Not a warning, a lock. Nobody can merge this, admins included.
- Where the rule lives

Settings, Rules, ruleset protect-main. Box 5 of the flowchart. Not in the YAML.

Fixed, both green, ready to merge

Summary

All jobs

1

✓ check (ubuntu-latest)

✓ check (windows-latest)

Run details

Usage

Workflow file

Triggered via pull request 1 minute ago

Greyisheep synchronize #1 [add-pr-gate](#)

Status

Success

pr-gate.yml

on: pull_request

Matrix: check

✓ 2 jobs completed

Show all jobs

- 1

Both legs green
Ubuntu and Windows agree, so the ruleset unlocks the merge button.
- **The one-line fix**
`REPORT = os.path.join("data", "reports", "summary.txt")`
Build the expected path the same way the code does.
- **Merge it live**
Pull request #1 is open and green right now. Merge it in front of the room.

From a laptop to a pipeline

First, how this app gets deployed by hand. Then the pipeline that replaces it, the one you will build yourselves before lunch.

YOU WILL BE ABLE TO

- Say what goes wrong when people deploy by hand
- Read a full deploy pipeline top to bottom
- Set up a repo so it can deploy with no password stored
- Fix four failures that every team hits

WATCH AND READ

- The template: github.com/Greyisheep/gh-200-team-app
- Its LAB.md, stage by stage
- The instructor's working copy: [gh-200-team-instructor](#)
- Docs: Deployment environments

BY HAND • FOR REAL, FROM MY LAPTOP

Five commands, and the first try failed

```
$ python3 scripts/stamp.py
$ docker build -t europe-west1-docker.pkg.dev/deepstack-492609/gh200-team-0/frontend:manual app
$ gcloud auth configure-docker europe-west1-docker.pkg.dev
$ docker push europe-west1-docker.pkg.dev/.../gh200-team-0/frontend:manual
$ gcloud run deploy gh200-team-0 --image .../frontend:manual --region europe-west1
ERROR: (gcloud.run.deploy) Revision 'gh200-team-0-00007-6pm' is not ready and cannot serve traffic. The user-provided container failed to start and listen on the port defined provided by the PORT=8080 environment variable within the allocated timeout.
$ docker build --platform linux/amd64 -t .../frontend:manual app # second try
Service [gh200-team-0] revision [gh200-team-0-00008-v82] has been deployed and is serving 100 percent of traffic.
```

Works on my machine

A Mac builds ARM images. Cloud Run runs AMD64. The container never started.

Nothing warned me

The build and the push both succeeded. Only the deploy, in production, found it.

My own admin login

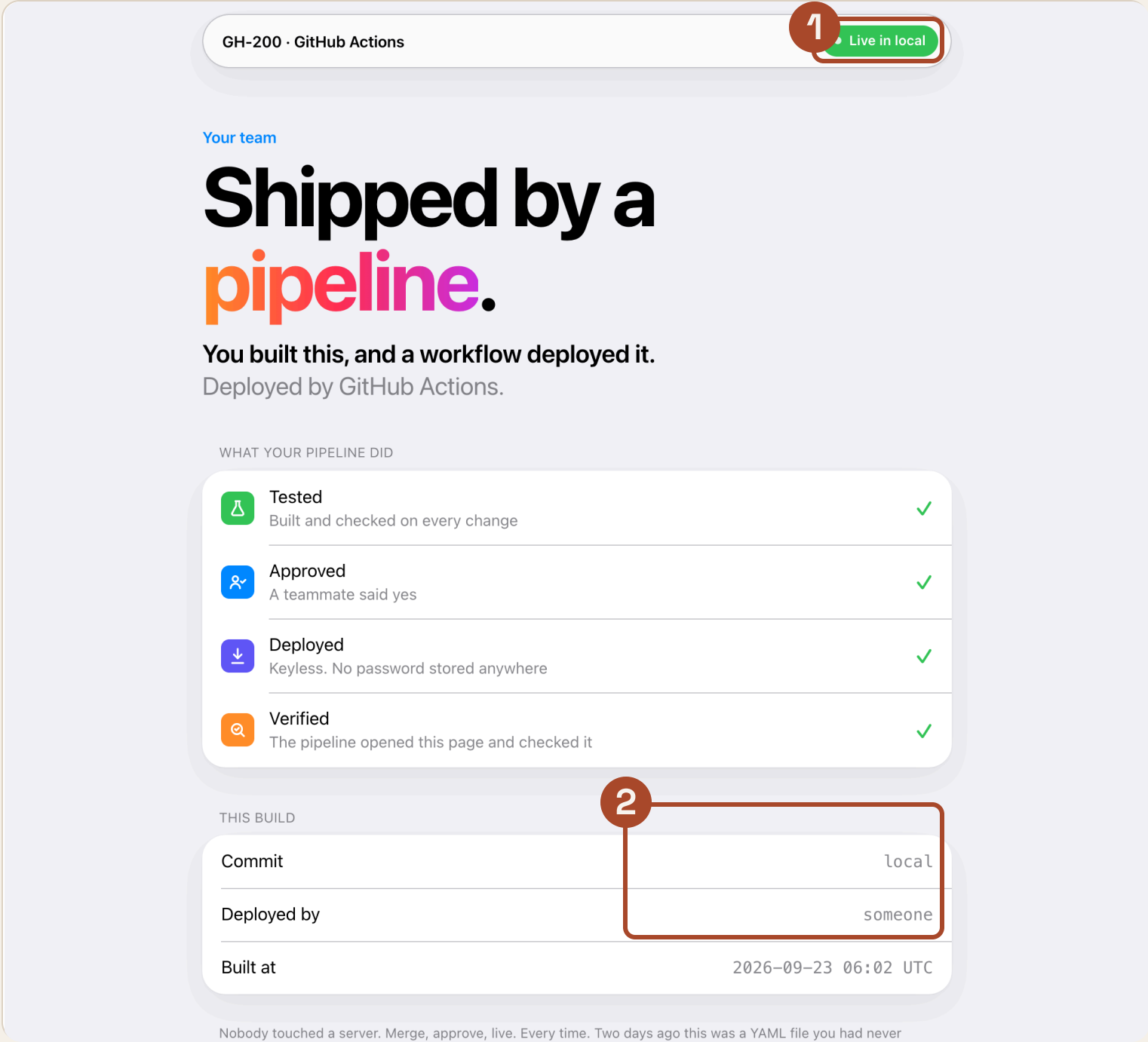
Whoever deploys by hand holds keys to the whole project, on a laptop.

Second try worked

Only because I knew the flag. The next person might not.

BY HAND · WHAT WENT LIVE

It is live. But what is live?



1 **Live in local**

The page does not even know which environment it is in.

2 **Commit local, deployed by someone**

Which commit is this? Who shipped it? There is no record, only my terminal history.

→ **Compare after lunch**

Your pipeline's page will say the exact commit, the person, the time, and production.

Real screen: <https://gh200-team-0-130785602363.europe-west1.run.app>, after the manual deploy

What the pipeline buys you

	By hand	The pipeline you build today
Who can deploy	Anyone with admin keys on a laptop	Only the pipeline, after a teammate approves
Which code	Whatever was on my disk	The exact commit merged to main, shown on the page
Tested first	If I remembered	Every time, the same checks
Which machine	My Mac, which built the wrong image	A clean Linux machine, every time
The record	My terminal history	The Actions tab, for good
The password	A long-lived admin login	None. A token that lives about an hour

The pipeline you will build, as boxes

TRIGGER

Every pull request and every push to main

`on: pull_request, push`

JOB 1 · TEST

Build the page into an image, run it, check it renders, pick a version tag

`outputs: tag`

JOB 2 · DEPLOY

Only on main. Waits for a teammate. Logs in with no password

`environment: production`

THEN

Build and push the image, deploy it, open the live page and check the commit

`Check it is live`

One file

Both jobs live in `.github/workflows/pipeline.yml`. The finished version is `reference/pipeline-final.yml` in the template, if you are ever stuck.

Two jobs, two machines

The test job's machine is thrown away before deploy starts. That fact is one of today's four failures.

You start with job 1

The template already has the test job. You add the deploy job, and it will fail in four instructive ways.

test: build it, run it, check it, name it

```
test:
  name: Build and smoke test
  runs-on: ubuntu-latest
  timeout-minutes: 10
  outputs:
    tag: ${ steps.version.outputs.tag }
  steps:
    - uses: actions/checkout@3d3c42e5... # v7.0.1
    - name: Stamp the page
      ...
    - name: Build the image
      run: docker build -t frontend:test app
    - name: Run it and check it answers
      ...
    - name: Pick the version tag
      id: version
      run: echo "tag=${GITHUB_SHA::7}-${GITHUB_RUN_NUMBER}" >> "$GITHUB_OUTPUT"
```

Stamp the page

Writes your team, commit and name into the HTML, from variables.

Build and run

The same Dockerfile production uses, started on the runner and checked with curl.

Pick the version tag

Commit plus run number, like **912e105-12**.
Written to **\$GITHUB_OUTPUT**.

outputs: tag

The job publishes the tag, so the deploy job can read it. Trap 3 is about this line.

deploy: only main, only after a yes, no password

```
deploy:
  name: Deploy to Cloud Run
  needs: test
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  timeout-minutes: 15
  environment:
    name: production
    url: ${ steps.deploy.outputs.url }
  concurrency:
    group: deploy-production
    cancel-in-progress: false
  permissions:
    contents: read
    id-token: write
  env:
    TAG: ${ needs.test.outputs.tag }
```

if:

Pull requests never deploy. Trap 4.

environment: production

The pause for a teammate, and the only place Google trusts. Trap 2.

permissions: id-token: write

Lets this job ask GitHub for a login token. Trap 1.

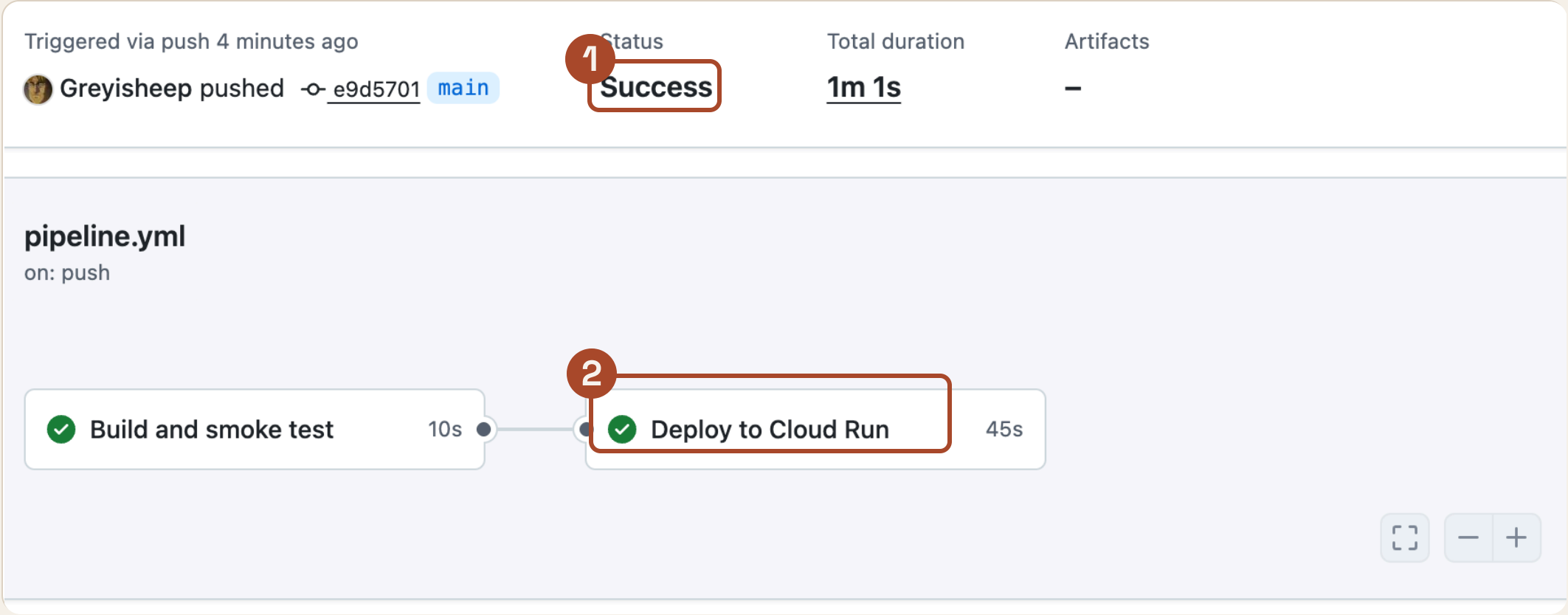
env: TAG from needs.test.outputs.tag

Reads the tag from the other job. Trap 3.

Then six steps

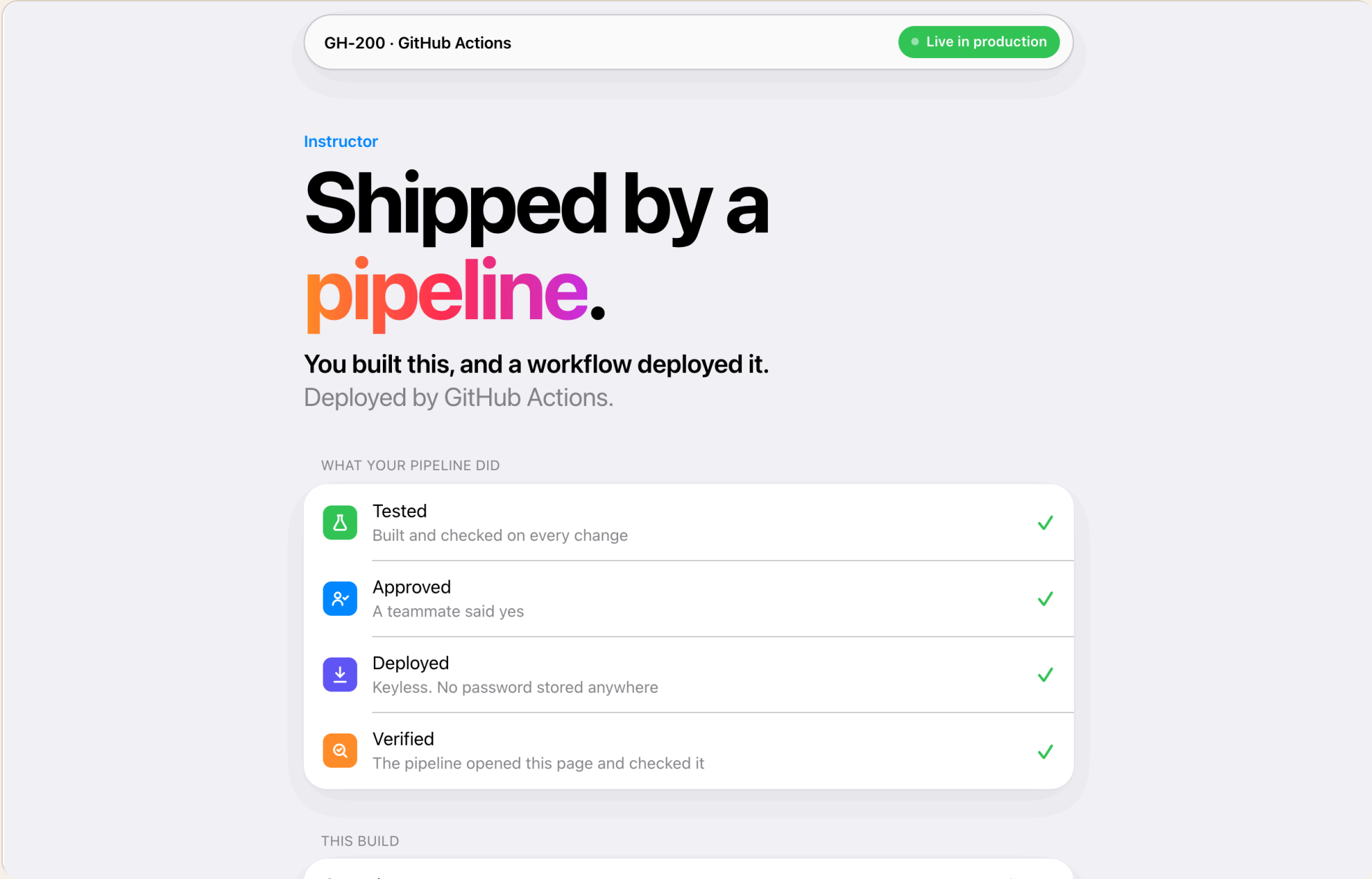
Stamp, keyless login, set up gcloud, build and push, deploy, check it is live.

Green, deployed, and checked



- 1 **Success**
Both jobs green. The deploy job ran only because this was a push to main.
- 2 **Deploy to Cloud Run**
It stamped, logged in, built, pushed, deployed and opened the live page itself.
- **The run summary**
Ends with the live URL and the tag. That tag is what a rollback needs.

The same page every team gets when theirs works



- 1 **Your team**
The TEAM variable, stamped in at deploy time.
- 2 **Four green ticks**
Tested, approved, deployed keyless, verified by the pipeline itself.
- 3 **This build**
The exact commit, who deployed it, and when. Compare the manual version.

What changes between a CI that works and one you would trust

	A CI that works	A CI you would trust
Triggers	on: [push]	PRs test, main deploys, a schedule for the slow stuff
Order	One big job	Cheap checks first, expensive ones only if those pass
Dependencies	Mocked, or skipped	A real database in a service container
Failure	Everything blocks, or nothing does	Some checks block, some only warn, on purpose
Time	No limit	timeout-minutes on every job
Deploys	Whenever, concurrently	One at a time, behind a gate, and verified afterwards
Actions	@v2, @master	Pinned to a SHA
Remote scripts	Carry on after an error	set -e, so a failure is red

Five habits in the file you will build

permissions:

Read-only by default. Each job asks for exactly what it needs.

```
contents: read
```

Pinned actions

Every uses: is a full commit SHA with the version in a comment.

```
@3d3c42e5 ... # v7.0.1
```

No stored passwords

Keyless login. The repo holds seven plain variables and zero secrets.

```
id-token: write
```

Untrusted text stays data

Nothing a person typed is pasted into run:.
Values go in through env:.

```
env: then $VAR
```

A human gate

Production only after a teammate approves,
and only from main.

```
environment: production
```

Check it yourself

Run **zizmor** on your workflow after lunch. The
reference file comes back clean.

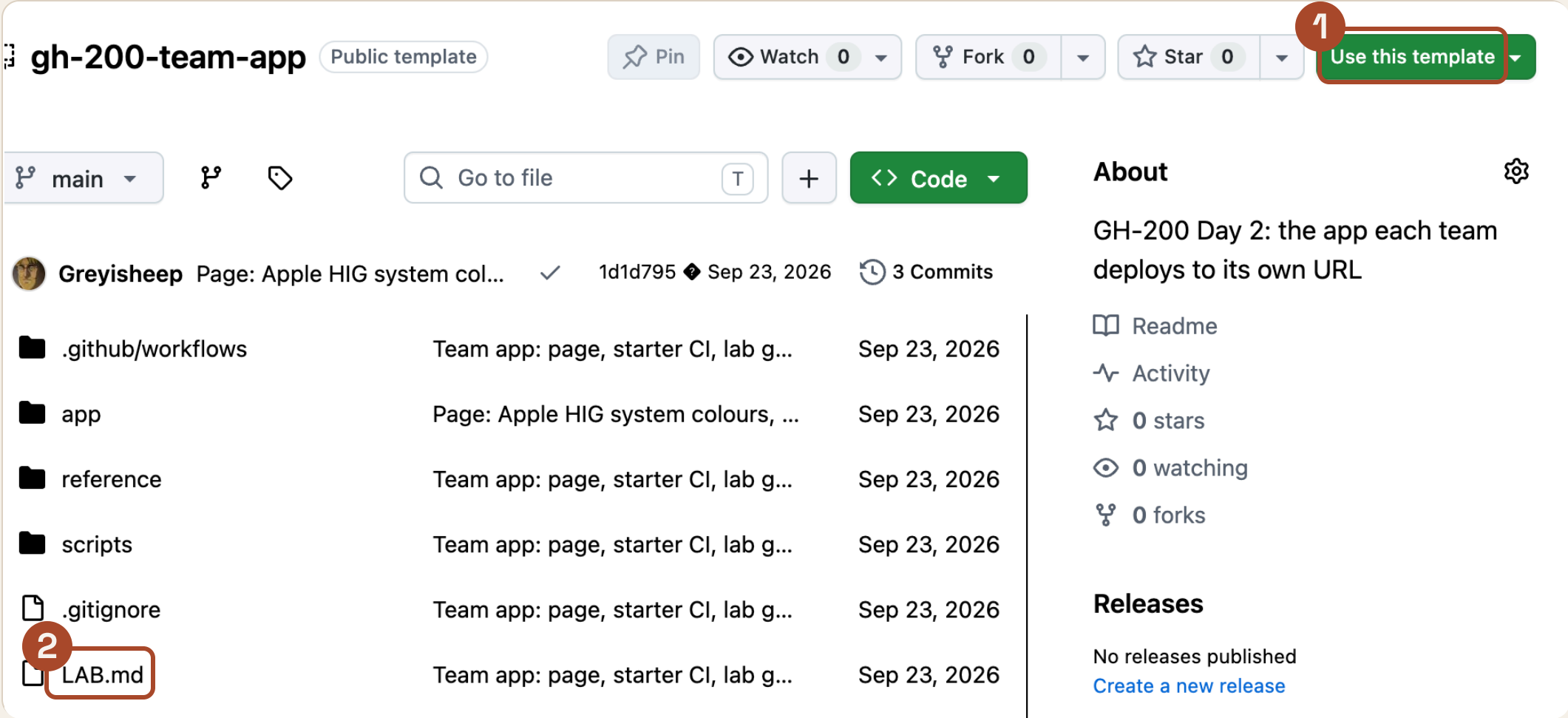
env, vars and secrets are not interchangeable

	env	vars	secrets
Set where	In the workflow file	Settings, at repo, environment or org	Settings, at repo, environment or org
Read with	env.NAME or \$NAME	vars.NAME	secrets.NAME
In Git history	Yes, it is in the file	No	No
In the logs	Shown	Shown, not masked	Masked as ***
Can you read it back?	Yes	Yes	Never. Replace or delete only
Use it for	Values for this run	Region, project ID, a banner	Passwords, tokens, keys

The test for a secret is not "is it sensitive". It is: **would I be in trouble if this appeared in a public log?**

STAGE 0 • YOUR OWN COPY

One click gives your team its own repo



- 1 Use this template**
Create a new repository. Owner: one person in the team. Public. Name: gh-200-team-<n>.
- 2 LAB.md**
Every step for today, including the four that fail on purpose.
- Then post the name**
Put your repo's full name in the class chat. I connect it to your team's cloud service.

Settings, Secrets and variables, Actions, Variables

1

SecretsVariables

Environment variables

This environment has no variables.

Manage environment variables

Repository variables

2

New repository variable

3

Name	Value	Last updated
AR_REPO	gh200-team-0	Sep 23, 2026
DEPLOYER_SA	gh200-team-0@deepstack...	Sep 23, 2026
GCP_PROJECT	deepstack-492609	Sep 23, 2026
GCP_REGION	europe-west1	Sep 23, 2026
SERVICE	gh200-team-0	Sep 23, 2026
TEAM	Instructor	Sep 23, 2026
WIF_PROVIDER	projects/130785602363/lo...	Sep 23, 2026

1 The Variables tab
Not Secrets. Nothing here is a password, so nothing needs hiding.

2 New repository variable
Seven times. Names exactly as in LAB.md, values from the chat.

3 What the pipeline reads
Project, region, service, image repo, the deploy account, the trust provider, your team name.

Settings, Environments, New environment: production

Environments / Configure production

Deployment protection rules

Configure reviewers, timers, and custom rules that must pass before deployments to this environment can proceed.

1

☒ Required reviewers

Specify people or teams that may approve workflow runs when they access this environment.

Add up to 5 more reviewers

Search for people or teams...

Greysisheep

×

☐ Prevent self-review

Require a different approver than the user who triggered the workflow run.

☐ Wait timer

Set an amount of time to wait before allowing deployments to proceed.

Enable custom rules with GitHub Apps

Preview

Learn about existing apps or create your own protection rules so you can deploy with confidence.

☒ Allow administrators to bypass configured protection rules

Save protection rules

2

Deployment branches and tags

Limit which branches and tags can deploy to this environment

Selected branches and tags

▼

1

Required reviewers

Add a teammate, not yourself. They will click Approve on every deploy.

2

Deployment branches

Selected branches, add main. Anything else is refused. Trap 4 depends on this.

→

Name it exactly

production, lower case. The pipeline and Google both look for that exact name.

Real screen: github.com/Greysisheep/gh-200-team-instructor

Four teams, four services, four URLs

Team	Your URL	Your deploy account can touch
Team 1	gh200-team-1-130785602363.europe-west1.run.app	Only service gh200-team-1 and its image repo
Team 2	gh200-team-2-130785602363.europe-west1.run.app	Only service gh200-team-2 and its image repo
Team 3	gh200-team-3-130785602363.europe-west1.run.app	Only service gh200-team-3 and its image repo
Team 4	gh200-team-4-130785602363.europe-west1.run.app	Only service gh200-team-4 and its image repo

Open yours now

Until your first deploy it shows a Cloud Run placeholder page. When your pipeline works, it turns into the page you just saw, with your team's name.

You cannot break each other

Each team's deploy account has rights on its own service only. Nothing at project level. Try deploying to another team's service: Google refuses.

The first deploy attempt. Paste it exactly

This is how most people write their first deploy job. It fails three times. That is the lesson.

a) at the end of the test job

```
- name: Pick the version tag
  run: echo "TAG=${GITHUB_SHA::7}-${GITHUB_RUN_NUMBER}" >> "$GITHUB_ENV"
```

b) the deploy job (full text in LAB.md)

```
deploy:
  name: Deploy to Cloud Run
  needs: test
  runs-on: ubuntu-latest
  timeout-minutes: 15
  steps:
    - uses: actions/checkout@...
    - name: Stamp the page
    - name: Log in to Google Cloud, keyless
    - uses: google-github-actions/setup-gcloud@...
    - name: Build and push the image
    - name: Deploy
    - name: Check it is live
```

Commit to main

In the browser is fine. Then open the Actions tab.

When it goes red

Click the red job. Read the red step. Write down what you think it means.

Then wait

We fix each one together, with the idea behind it. Three reds before a green.

Red 1: the login step cannot even start

Triggered via push 16 minutes ago

Greyisheep pushed 7768784 main

Status

Failure

Total duration

16s

Artifacts

-

pipeline.yml

on: push

✓ Build and smoke test

6s

✗ Deploy to Cloud Run

5s

Annotations

1 error and 2 notices

✗ 1 Deploy to Cloud Run

google-github-actions/auth failed with: gitHub Actions did not inject \$ACTIONS_ID_TOKEN_REQUEST_TOKEN or \$ACTIONS...

Show more

- 1

The error, in full

google-github-actions/auth failed with: gitHub Actions did not inject \$ACTIONS_ID_TOKEN_REQUEST_TOKEN or \$ACTIONS_ID_TOKEN_REQUEST_URL into this job. This most likely means the GitHub Actions workflow permissions are incorrect.
- Before the next slide

Ask the room: what does it think is missing? Get two guesses.

The job was never allowed to ask for a token

```
permissions:  
  contents: read  
  id-token: write
```

Two different tokens

GITHUB_TOKEN talks to GitHub. The OIDC token proves to Google who this job is. The job must ask for it.

Why it was missing

The top of the file says **permissions: contents: read**. Every job gets only that, unless it asks for more.

The habit

Least privilege by default, raise it per job. Only deploy gets id-token: write.

Red 2: Google says no

Triggered via push 15 minutes ago

 Greyisheep pushed

 `b0fe2e5`

`main`

Status

Failure

Total duration

38s

Artifacts

—

pipeline.yml

on: push

 Build and smoke test

30s

 Deploy to Cloud Run

3s

Annotations

1 error and 2 notices

 1 Deploy to Cloud Run

google-github-actions/auth failed with: failed to generate Google Cloud federated token for //iam.googleapis.com/...

Show more

- 1

The error, in full

failed to generate Google Cloud federated token ...
{"error":"unauthorized_client","error_description":"The given credential is rejected by the attribute condition."}
- Before the next slide

Ask the room: what does it think is missing? Get two guesses.

COMMIT · FIX 2: DEPLOY FROM THE PRODUCTION ENVIRONMENT

Google only trusts the production environment

```
environment: production
```

What Google checks

`assertion.repository` in [your repo] &&
`assertion.environment=='production'`. That is the real rule, set up for your team.

No environment, no trust

The token said which repo, but not production.
Rejected, before anything reached the cloud.

And now it waits

The environment also means your teammate must approve. The next run pauses for them.

Red 3: an image with no name

Triggered via push 14 minutes ago

 Greyisheep pushed

 7aa5875

main

Status

Failure

Total duration

30s

Artifacts

—

pipeline.yml

on: push

✓

Build and smoke test

8s

✗

Deploy to Cloud Run

16s

Annotations

1 error and 2 notices

✗

1

Deploy to Cloud Run

Process completed with exit code 1.

- 1

The error, in full

ERROR: failed to build: invalid tag "europe-west1-docker.pkg.dev/deepstack-492609/gh200-team-0/frontend": invalid reference format
- Before the next slide

Ask the room: what does it think is missing? Get two guesses.

Every job is a new machine. Hand the value over on purpose

```
outputs:
  tag: ${ steps.version.outputs.tag }
...
- name: Pick the version tag
  id: version
  run: echo "tag=..." >> "$GITHUB_OUTPUT"
...
env:
  TAG: ${ needs.test.outputs.tag }
```

Why TAG was empty

\$GITHUB_ENV only lasts for the rest of the same job.
Deploy runs on a different machine.

The relay

Step writes **tag=** to \$GITHUB_OUTPUT, the step has **id: version**, the job publishes **outputs: tag**, deploy reads **needs.test.outputs.tag**.

Yesterday's build_id file

The exact same shape. version is the step id, tag is the name we chose.

Red 4: a pull request tried to deploy

Triggered via pull request 11 minutes ago

Status

Total duration

Artifacts

Greyisheep opened #1 `new-footer`

Failure

12s

—

1 pipeline.yml

on: pull_request

✓ Build and smoke test

8s

✗ Deploy to Cloud Run

1s

Annotations

2 errors and 1 notice

2 Deploy to Cloud Run

Branch "refs/pull/1/merge" is not allowed to deploy to production due to environment protection rules.

- 1 on: pull_request
This run came from a pull request, not a push to main.
- 2 The error, in full
Branch "refs/pull/1/merge" is not allowed to deploy to production due to environment protection rules.

COMMIT • FIX 4: PULL REQUESTS TEST, ONLY MAIN DEPLOYS

Pull requests test. Only main deploys

```
if: github.event_name == 'push' && github.ref == 'refs/heads/main'
```

Why it failed

The deploy job ran for the pull request too. The environment allows main only, so GitHub refused.

The fix

An if: on the deploy job. On a pull request it now shows as skipped, not red.

Two layers

The if: says what we intend. The branch rule enforces it even if someone edits the if:.

AFTER STAGE 3 • YOUR ACTIONS TAB

Red, red, red, green. Then a pull request

All workflows

Showing runs from all workflows

11 workflow runs

WorkflowEventStatusBranchActor

| ✓ New page design Pipeline #10: Commit e9d5701 pushed by Greyisheep main 3 minutes ago 1m 1s ... |
| ✓ Rollback Rollback #1: Manually run by Greyisheep main 7 minutes ago 27s ... |
| ✓ Stage 5: add a Rollback workflow Pipeline #9: Commit b109be9 pushed by Greyisheep main 8 minutes ago 57s ... |
| 3 ✓ Change the footer (#1) Pipeline #8: Commit d15281b pushed by Greyisheep main 10 minutes ago 1m 0s ... |
| ✓ Change the footer Pipeline #7: Pull request #1 synchronize by Greyisheep new-footer 10 minutes ago 9s ... |
| ✗ Change the footer Pipeline #6: Pull request #1 opened by Greyisheep new-footer 11 minutes ago 12s ... |
| 2 ✓ Fix 3: hand the tag over with job outputs Pipeline #5: Commit c0935e0 pushed by Greyisheep main 13 minutes ago 1m 12s ... |
| ✗ Fix 2: deploy from the production environment Pipeline #4: Commit 7aa5875 pushed by Greyisheep main 14 minutes ago 30s ... |
| ✗ Fix 1: let the deploy job ask for an OIDC token Pipeline #3: Commit b0fe2e5 pushed by Greyisheep main 15 minutes ago 38s ... |
| 1 ✗ Stage 2: first deploy attempt Pipeline #2: Commit 7768784 pushed by Greyisheep main 16 minutes ago 16s ... |
| ✓ Initial commit main 16 minutes ago 1m 1s ... |

- 1

Stage 2: first deploy attempt

Red 1. Then Fix 1 red, Fix 2 red: each fix uncovers the next problem.
- 2

Fix 3

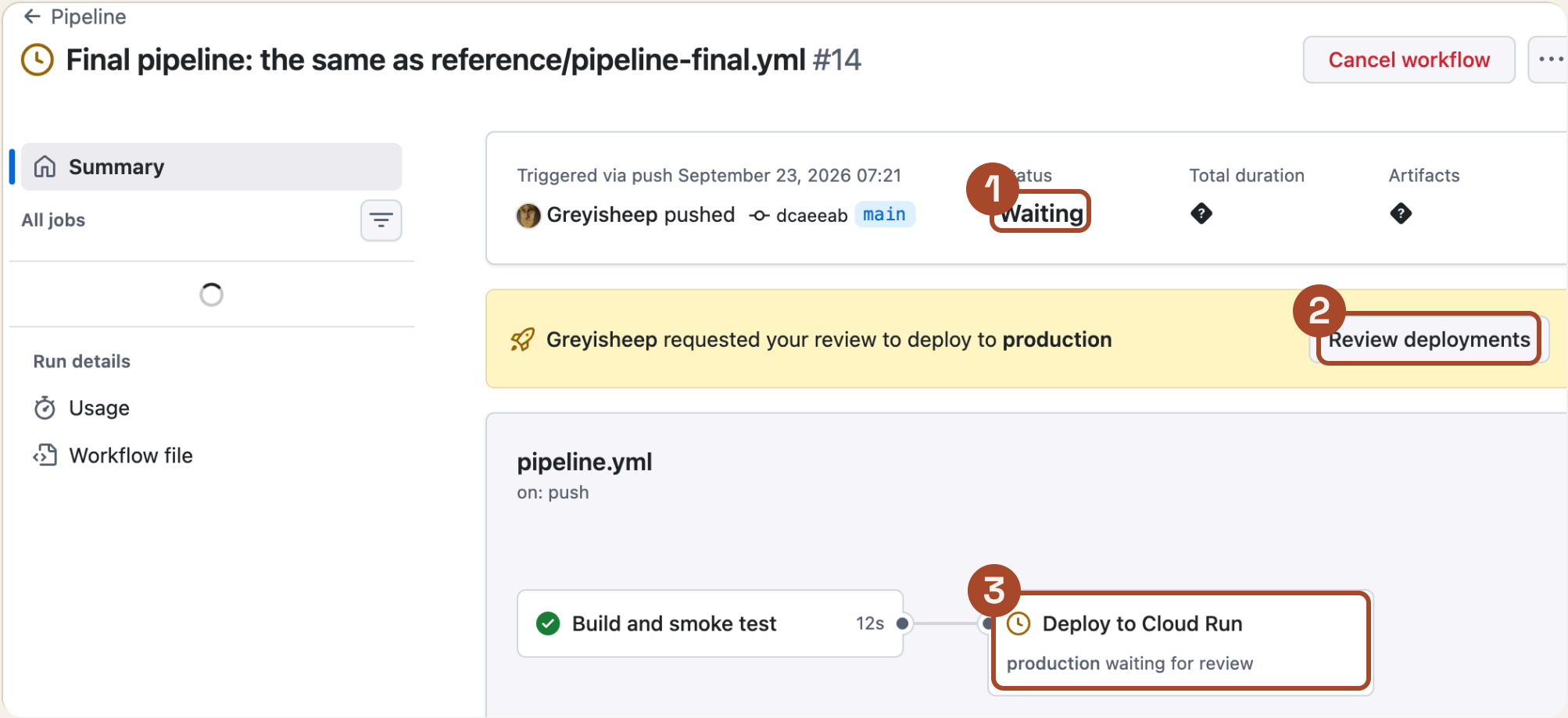
The first green deploy. The team page is live.
- 3

Change the footer (#1)

The pull request, fixed with Fix 4, then merged and deployed.

STAGE 4 • A TEAMMATE SAYS YES

Every deploy now waits for a person



- 1 Waiting**
Not failed. Paused, costing nothing, until someone approves.
- 2 Review deployments**
Your teammate clicks it, ticks production, writes why, and approves.
- 3 Deploy to Cloud Run**
Runs the moment they approve. Then refresh your team's URL.

How the new version reaches users

Pattern	How it works	Rolling back	Costs
Recreate	Stop the old version, start the new one	Redeploy the old one. Minutes, with downtime	Nothing extra. Downtime
Rolling	Replace servers or pods a few at a time	Roll the other way. Minutes	Old and new run side by side briefly
Blue/green	A full second copy, then switch all traffic at once	Switch back. Seconds	Twice the servers while it runs
Canary	A small share of users gets it first	Send that share back to zero. Seconds	Needs traffic splitting and watching
Feature flag	Code ships switched off, turned on per user	Flip the flag. No deploy at all	Flags to clean up later

Cloud Run gives you blue/green and canary for free: every deploy is a new **revision**, and traffic is a setting you can move between revisions.

Put an older build back, with one input

1

Manually triggered 8 minutes ago

Status

Success

Total duration

27s

Artifacts

-

Greyisheep

b109be9

main

rollback.yml

on: workflow_dispatch

2

rollback

21s

https://gh200-team-0-cu5qickhka-ew....

- 1

Manually triggered

Actions, Rollback, Run workflow. One box: the tag, like c0935e0-5.
- 2

The live URL

The old commit was back on the page in 27 seconds. Same approval as a deploy.
- **Where tags come from**

Each deploy's run summary ends with its tag. The image for every tag is still in the registry.

The same idea when you deploy over SSH

```
- uses: appleboy/ssh-action@<SHA> # v1
  env:
    SHA: ${ inputs.sha }           # validated earlier
  with:
    host: ${ vars.DEPLOY_HOST }
    username: deploy
    key: ${ secrets.DEPLOY_SSH_KEY }
    envs: SHA
    script: |
      set -euo pipefail
      cd /srv/app
      export IMAGE_TAG="$SHA"
      docker compose pull api
      docker compose up -d api
      curl -fsS --retry 5 localhost:8000/health
```

Tag images by SHA

The build job pushes api:<sha> to a registry, instead of building on the server.

compose reads the tag

image: registry/api:\${IMAGE_TAG} in docker-compose.yml.
Deploy and rollback are the same script.

envs: passes the value

The SHA reaches the server as a variable, never pasted into the script text.

Keep the last few images

Prune old ones on a schedule, but never the last five. That is your rollback window.

15:00 • AFTER LUNCH

Finish it, prove it, then roll it back

1

Stage 3 and 4

If you have not yet: open a pull request, see red 4, fix it, merge, get a teammate to approve.

2

Prove it

Your team's URL shows your team, your commit and your name. Screenshot it and post it.

3

Change something

Set BANNER, or edit the page. Merge it. Watch it go out without touching anything else.

4

Stage 5, stretch

Copy reference/rollback.yml into .github/workflows/. Deploy twice, then roll back to the older tag.

5

Check it

Run zizmor on your workflows. Compare with reference/pipeline-final.yml line by line.

One long one, with screenshots

- Twenty-eight questions, from both days
- Eight are real screens from today: the red runs, the locked merge, the waiting deploy
- Five are fill in the blank: the exact words from your pipeline
- Answers are spread across all four colours. No pattern to spot

The screens in the questions are the ones from today's slides. If you were watching, you have seen them.

GH-200 • TWO DAYS

You can now build the pipeline, not just read it

This week

Run zizmor on one real workflow at work.
Count what it finds. Fix the easiest three.

This month

Give one real deploy a Rollback workflow,
and run it once on a quiet day.

For the exam

GH-200 tests authoring, consuming,
extending and managing Actions at org
level. You have touched all four.

Slides, cheatsheet and every task with its answers: the student pack repo.

A workflow, top to bottom

```
name: CI                                # shown in the Actions tab
on:                                     # WHEN
  pull_request:
    branches: [main]
  push:
    branches: [main]
  workflow_dispatch:                    # the Run button
permissions:
  contents: read                        # token ceiling
jobs:
  test:                                # job id, you choose it
    runs-on: ubuntu-latest              # WHERE
    timeout-minutes: 10
    strategy:                            # HOW MANY
      fail-fast: false
      matrix:
        python: ['3.12', '3.13']
    steps:                              # WHAT
      - uses: actions/checkout@<SHA> # v7
      - uses: actions/setup-python@<SHA>
        with:
          python-version: ${{ matrix.python }}
      - run: pytest
```

The five questions

When: **on**. Where: **runs-on**. How many: **strategy**. What: **steps**.
Who enforces: **Settings, Rules**, not the file.

Where the file lives

.github/workflows/anything.yml, on the default branch for the Run button and schedules.

run: or uses:

run types a shell command. uses borrows an action, and with: feeds it inputs.

Job to job

needs: orders jobs. It never moves files: artifacts do. Every job is a new machine.

Quote these

Version numbers like '3.10'. Cron strings. Anything starting with * or @.

Chains, outputs and dollar signs

Chain	Read it as	Works in
steps.ID.outputs.NAME	The step with this id, its output	Later steps, same job
needs.JOB.outputs.NAME	A job I need, its published output	Jobs that list JOB in needs
needs.JOB.result	success, failure, cancelled, skipped	Jobs that list JOB in needs
matrix.KEY	This copy's value	Jobs with a matrix
github.event...	The event that started the run	Anywhere. Untrusted text
vars.X, secrets.X, env.X	Settings values, and env	secrets not in job-level if

The relay

Step: **echo "n=5" >> \$GITHUB_OUTPUT** with **id: s**. Job: **outputs: n: \${{ steps.s.outputs.n }}**. Other job: **needs: job** then **\${{ needs.job.outputs.n }}**.

Two dollars

\${{ x }} GitHub fills in before the step runs. **\$X** the shell reads while it runs. Untrusted text: put it in env:, read it as \$X.

Special files

GITHUB_OUTPUT outputs. GITHUB_ENV env from the next step. GITHUB_STEP_SUMMARY Markdown on the run page. Always >>.

The production checklist

Triggers

PRs test. Push to main deploys, with **if:**. Slow stuff on a schedule.

Order

Lint and unit tests first. Build and integration after. **needs:** links them.

Token

permissions: contents: read at the top. Raise per job only.

Actions

Full SHA, version in a comment, Dependabot keeps them fresh.

Injection

No `${{ github.event... }}` inside `run:`. Use `env:`.

Time

timeout-minutes on every job. Default is 6 hours.

Concurrency

Cancel superseded CI. Never cancel a deploy, queue it.

Deploy

Environment with reviewers. Migrate, then swap. **set -e**. Exact commit.

Verify

Hit the live URL after deploy, and fail if it is wrong.

Speed

Cache dependencies. paths filters. One matrix leg on PRs, the full one nightly.

Checks

actionlint for mistakes, zizmor for security, CodeQL for workflows.

Rollback

Images tagged by SHA. A Rollback workflow behind the same gate. Additive migrations only.